[BUG] Issue with the gradients of the expectation value of an operator

https://github.com/PennyLaneAI/pennylane/issues/4199

ROW 7

[BUG] Issue with the gradients of the expectation value of an operator #4199

New issue

Closed #4251

jackaraz opened on Jun 1, 2023

Expected behavior

Hi, I'm trying to compute the gradient of the circuit with respect to the parameters on the operator $\theta_{i,i+1} (\sigma^+*i\sigma^-*{i+1} + \sigma^+_{i+1}\sigma^-_i)$ where θ is my variable. When I compute the expectation value of the circuit, I can see that individual terms of my operator are differentiable, e.g. for $\theta_0(\sigma_0^+\sigma_1^- + \sigma_1^+\sigma_0^-)$ I can get a gradient with respect to θ_0 but as soon as I add other terms I get None which I was expecting to get the gradient for every term.

Actual behavior

Gradient returns `None`.

Additional information

I get zero for the gradient when I use autograd or Jax.

Source code

```
import pennylane as qml
import tensorflow as tf
import pennylane.numpy as np

# here s_prod or sum is not necessary +, @ and * works the same
sig_plus = lambda w: qml.s_prod(0.5, (qml.sum(qml.PauliX(w) , qml.s_prod(1j, qml.PauliY(w)))))
sig_minus = lambda w: qml.s_prod(0.5, (qml.sum(qml.PauliX(w) , qml.s_prod(-1j, qml.PauliY(w)))))
term = lambda w: qml.sum(qml.prod(sig_plus(w) , sig_minus(w + 1)) , qml.prod(sig_plus(w + 1), sig_minus(w)))

@qml.qnode(qml.device("default.qubit.tf", wires=4), interface="tf")
def circ(phi, theta):
    # This portion of the circuit is random not important
    qml.RY(phi[0], 0)
    qml.RY(phi[1], 1)
    qml.RY(phi[2], 2)
    qml.CNOT([0,1])
    qml.CNOT([1,2])
    ##########################
    return qml.expval(theta[0]*term(0) + theta[1]*term(1) ) # only 0 or only 1 works just fine, also tried wit

theta = tf.Variable(np.random.uniform(0,1,3))
phi = tf.Variable(np.random.uniform(0,1,3))

def cost(phi, theta):
    return circ(phi, theta)

with tf.GradientTape() as tape:
    loss = cost(phi, theta)
tape.gradient(loss, [phi, theta])

#[<tf.Tensor: shape=(3,), dtype=float64, numpy=array([0.00131378, 0.00458278, 0.00750094])>,
# None]
```

Tracebacks

```
No error has been generated
```

System information

```
TF version 2.11.0
TF compiler version Apple LLVM 13.1.6 (clang-1316.0.21.1)

Name: PennyLane
```

Assignees

timmysilv

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

support trainable Sum observables
PennyLaneAI/pennylane

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

Tracebacks

```
No error has been generated
```

System information

```
TF version 2.11.0
TF compiler version Apple LLVM 13.1.6 (clang-1316.0.21.1)

Name: PennyLane
Version: 0.30.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author:
Author-email:
License: Apache License 2.0
Location: /Users/jackaraz/packages/miniconda3/lib/python3.9/site-packages
Requires: appdirs, autograd, autoray, cachetools, networkx, numpy, pennylane-lightning, requests, rustworkx, s
Required-by: field-theory, PennyLane-Lightning, PennyLane-qiskit, qhbm

Platform info:           macOS-13.3.1-arm64-arm-64bit
Python version:          3.9.16
Numpy version:           1.23.5
Scipy version:           1.10.0
Installed devices:
- default.gaussian (PennyLane-0.30.0)
- default.mixed (PennyLane-0.30.0)
- default.qubit (PennyLane-0.30.0)
- default.qubit.autograd (PennyLane-0.30.0)
- default.qubit.jax (PennyLane-0.30.0)
- default.qubit.tf (PennyLane-0.30.0)
- default.qubit.torch (PennyLane-0.30.0)
- default.qutrit (PennyLane-0.30.0)
- null.qubit (PennyLane-0.30.0)
- lightning.qubit (PennyLane-Lightning-0.30.0)
- qiskit.aer (PennyLane-qiskit-0.28.0)
- qiskit.basicaer (PennyLane-qiskit-0.28.0)
- qiskit.ibmq (PennyLane-qiskit-0.28.0)
- qiskit.ibmq.circuit_runner (PennyLane-qiskit-0.28.0)
- qiskit.ibmq.sampler (PennyLane-qiskit-0.28.0)
```

Existing GitHub issues

- [x] I have searched existing GitHub issues to make sure the issue does not already exist.

jackaraz added bug 🦠 on Jun 1, 2023

jackaraz on Jun 1, 2023 · edited by jackaraz Edits Contributor Author

Here is a more concrete example of what I was expecting to happen:

```python
import pennylane as qml
import jax

PRNG  = jax.random.PRNGKey(3)

def sigma_plus(wire: int):
    return qml.s_prod(0.5, (qml.sum(qml.PauliX(wire) , qml.s_prod(1j, qml.PauliY(wire)))))
def sigma_minus(wire: int):
    return qml.s_prod(0.5, (qml.sum(qml.PauliX(wire) , qml.s_prod(-1j, qml.PauliY(wire)))))
def hopping_term(wire: int):
    return qml.sum(qml.prod(sigma_plus(wire) , sigma_minus(wire + 1)) , qml.prod(sigma_plus(wire + 1), sigma_m
def hopping_terms(param: tf.Tensor, wires: list[int]):
    return qml.sum(*[qml.s_prod(param[idx], hopping_term(idx)) for idx in wires])

theta = jax.random.uniform(PRNG, (3,), minval=0, maxval=1)
phi = jax.random.uniform(PRNG, (4,), minval=0, maxval=1)

@qml.qnode(qml.device("default.qubit", wires=4), interface="jax")
def circ(phi, hamil):
```

Microsoft Tea

Here is a more concrete example of what I was expecting to happen:

```
import pennylane as qml
import jax

PRNG  = jax.random.PRNGKey(3)

def sigma_plus(wire: int):
    return qml.s_prod(0.5, (qml.sum(qml.PauliX(wire) , qml.s_prod(1j, qml.PauliY(wire)))))
def sigma_minus(wire: int):
    return qml.s_prod(0.5, (qml.sum(qml.PauliX(wire) , qml.s_prod(-1j, qml.PauliY(wire)))))
def hopping_term(wire: int):
    return qml.sum(qml.prod(sigma_plus(wire) , sigma_minus(wire + 1)) , qml.prod(sigma_plus(wire + 1), sigma_m
def hopping_terms(param: tf.Tensor, wires: list[int]):
    return qml.sum(*[qml.s_prod(param[idx], hopping_term(idx)) for idx in wires])

theta = jax.random.uniform(PRNG, (3,), minval=0, maxval=1)
phi = jax.random.uniform(PRNG, (4,), minval=0, maxval=1)

@qml.qnode(qml.device("default.qubit", wires=4), interface="jax")
def circ(phi, hamil):
    qml.RY(phi[0], 0)
    qml.RY(phi[1], 1)
    qml.RY(phi[2], 2)
    qml.RY(phi[3], 3)
    qml.CNOT([0,1])
    qml.CNOT([1,2])
    qml.CNOT([2,3])
    return qml.expval(hamil)

def cost(phi, theta, term_index: int):
    Hamil = theta[term_index] * hopping_term(term_index)
    return  circ(phi, Hamil)

grad = jax.grad(cost, argnums=[0,1])

sum(grad(phi, theta, idx)[1] for idx in range(3))
# Array([0.03910499, 0.05563865, 0.04233078], dtype=float64))
```

Here since each term is independent in the gradient, I am summing the grad_theta vectors; each vector has 3 elements, and all the elements except `term_index` are zero. This example should work just as is with the `hopping_terms` function; however, it just gives me zero. Am I interpreting this wrong?

isaacdevlugt on Jun 2, 2023 · edited by isaacdevlugt Edits Contributor ···

Hey @jackaraz! I think this might be a better minimal example that's at the heart of the issue:

```
import pennylane as qml
from pennylane import numpy as np

from autograd import grad

H = lambda x : x * qml.PauliZ(0)

dev = qml.device("default.qubit", wires=1)

@qml.qnode(dev)
def circuit(x):
    qml.Hadamard(0)
    return qml.expval(H(x))

x = np.array([0.1], requires_grad=True)

grad(circuit, argnum=0)(x)

'''
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[7], line 12
      8     return qml.expval(H(x))
     10 x = np.array([0.1], requires_grad=True)
---> 12 grad(circuit, argnum=0)(x)

File [~/pennylane-torch/lib/python3.9/site-packages/autograd/wrap_util.py:20](https://file+.vscode-resource.vs
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[7], line 12
      8     return qml.expval(H(x))
     10 x = np.array([0.1], requires_grad=True)
---> 12 grad(circuit, argnum=0)(x)

File [~/pennylane-torch/lib/python3.9/site-packages/autograd/wrap_util.py:20](https://file+.vscode-resource.vs
     18 else:
     19     x = tuple(args[i] for i in argnum)
---> 20 return unary_operator(unary_f, x, *nary_op_args, **nary_op_kwargs)

File [~/pennylane-torch/lib/python3.9/site-packages/autograd/differential_operators.py:28](https://file+.vscoc
     21 @unary_to_nary
     22 def grad(fun, x):
     23     """
     24     Returns a function which computes the gradient of `fun` with respect to
     25     positional argument number `argnum`. The returned function takes the same
     26     arguments as `fun`, but returns the gradient instead. The function `fun`
     27     should be scalar-valued. The gradient has the same type as the argument."""
---> 28     vjp, ans = _make_vjp(fun, x)
     29     if not vspace(ans).size == 1:
     30         raise TypeError("Grad only applies to real scalar-output functions. "
     31                         "Try jacobian, elementwise_grad or holomorphic_grad.")
...
---> 55     if not op.is_hermitian:
     56         warnings.warn(f"{op.name} might not be hermitian.")
     58     return ExpectationMP(obs=op)

AttributeError: 'ArrayBox' object has no attribute 'is_hermitian'
'''
```

But if I switch the order of multiplication in the definition of `H`, everything magically works:

```
H = lambda x : qml.PauliX(0) * x

dev = qml.device("default.qubit", wires=1)

@qml.qnode(dev)
def circuit(x):
    qml.Hadamard(0)
    return qml.expval(H(x))

x = np.array([0.1], requires_grad=True)

grad(circuit, argnum=0)(x)

'''
array([1.])
'''
```

Indeed,

```
print(type(0.5*qml.PauliZ(0)))
print(type(0.5*qml.PauliZ(0)))
'''
<class 'pennylane.ops.qubit.hamiltonian.Hamiltonian'>
<class 'pennylane.ops.qubit.hamiltonian.Hamiltonian'>
'''
```

So, looks like there's a bug!

isaacdevlugt assigned trbromley and timmysilv and unassigned trbromley on Jun 2, 2023

isaacdevlugt on Jun 2, 2023 · edited by isaacdevlugt Edits Contributor

Just putting for consistency here too that if you instead use an explicit Hamiltonian constructor then everything works fine as well: `qml.Hamiltonian([x], [qml.PauliX(0)])`.